

TAHOE: Task-Aware Language Harness for Orchestrated Execution

Anonymous

Abstract

TAHOE is a structured reasoning language that teaches large language models (LLMs) to think in typed protocols—`COMPUTE`, `SELECT`, `DEDUCE`—rather than free-form chain-of-thought. We evaluate TAHOE as a thinking skill (system prompt) against classic inference on 10 public benchmarks (GSM8K, ARC, BBH, MMLU, LSAT, RACE) using GLM-5.3-Flash with 10 samples per benchmark, 3 trials per sample, 600 trials total. Results show TAHOE improves pass rate by 1% (89% vs. 88%) while reducing reasoning token consumption by 40% (60,055 vs. 100,595 output tokens). The harmonic mean of quality and efficiency is 1.250 (TAHOE) vs. 0.941 (classic), indicating TAHOE wins on both axes simultaneously. We provide formal operational semantics for the language, a frozen evaluation protocol, and bootstrap confidence intervals for all reported metrics.

1 Introduction

LLM reasoning quality varies with problem structure. Free-form chain-of-thought (CoT) prompting [24] improves accuracy but produces verbose, unstructured output that wastes tokens. We evaluate on ten public benchmarks — GSM8K [4], ARC [3], BBH [22], MMLU [11], LSAT-AR, and RACE [13] — and position the result against the agent-harness and token-efficiency literature (Section 5). As models are deployed at scale, the cost of reasoning tokens becomes a first-order concern: each redundant token increases latency, compute, and inference cost without improving answer quality.

TAHOE addresses this by providing a language for structured reasoning that the model applies internally as a *thinking skill*—a system prompt that shapes how the model reasons without changing the task or the output format. The key insight is that the system prompt (93 tokens) is a fixed infrastructure cost, while the model’s reasoning (output tokens) is where TAHOE delivers value: structured reasoning is more compact than free-form narrative, producing 40% fewer tokens while also improving accuracy.

Contributions.

- **TAHOE language:** a typed, protocol-based reasoning language with formal operational semantics (Section 2).
- **Evaluation methodology:** a frozen evaluation protocol with rigorous token accounting and bootstrap confidence intervals (Section 3).
- **Empirical results:** 600-trial controlled experiment showing 40% token reduction with 1% quality improvement (Section 4).
- **Formal semantics:** small-step operational semantics with progress and preservation theorems (Section 2.4).

2 TAHOE Language

TAHOE is a structured reasoning language that constrains LLM output into typed, protocol-governed patterns. The language has three core components: typed refs, protocols, and a

need-trigger mechanism.

2.1 Typed Refs

TAHOE distinguishes knowledge states using typed references (refs). Each ref carries a type tag from a fixed vocabulary:

Type	Role
G	Goal
C	Constraint
E	Evidence
H	Hypothesis
U	Unknown
O	Option
K	Criteria
D	Decision
P	Result
V	Verified
OUT	Output

This type system prevents assumptions from silently becoming facts and forces the model to commit to intermediate values. The types form a subtyping lattice (Section 2.4) where stronger types (e.g., **V** for verified) can substitute for weaker ones (e.g., **H** for hypothesis) but not vice versa.

2.2 Protocols

Three core protocols map task types to reasoning patterns:

- **Compute**: **E.givens** $\rightarrow$ steps with concrete numbers $\rightarrow$ **verify** $\rightarrow$ **answer**
- **Select**: **O.options** $\rightarrow$ eliminate wrong $\rightarrow$ **D.pick** $\rightarrow$ **answer**
- **Deduce**: **C.constraints** $\rightarrow$ assign positions explicitly ($1..N$) $\rightarrow$ **verify all** $\rightarrow$ **answer**

Protocols are composable: a complex task may invoke multiple protocols in sequence or in parallel. Counterfactual reasoning, for instance, reduces to three existing commands—**hypothesize**, **challenge**, **verify**—without requiring new syntax. The typed-ref system (**H.*** vs. **E.*** vs. **V.***) enforces the distinction between counterfactual hypothesis, factual evidence, and verified verdict, preventing the silent substitution that Pearl’s do-calculus was designed to expose [19].

2.3 Need-Trigger

TAHOE is need-triggered: the model applies structure only where it prevents errors. Simple tasks receive direct answers; multi-step tasks receive protocol structure. A *Rule 15* defines when to use Fast, Standard, or Critical mode based on task complexity signals (number of constraints, depth of reasoning required, presence of conflicting evidence).

This design avoids the overhead of structure on easy problems while ensuring structure is present on hard ones—a property that contributes to token efficiency without sacrificing quality.

2.4 Formal Semantics

We provide small-step operational semantics for TAHOE. A configuration is a triple $\langle P, \sigma, E \rangle$ where P is the program, σ is the committed state (a typed graph), and E is the append-only event log. Transitions have the form $\langle P, \sigma, E \rangle \rightarrow \langle P', \sigma', E' \rangle$.

State model. A state $\sigma = (N, L)$ is a typed graph where N is a set of typed nodes (each carrying a type $\tau(n)$, identifier, value, and revision) and L is a set of typed links (supporting, contradicts, depends_on, etc.). The committed state is a deterministic projection of the event log: $\sigma = \text{project}(\sigma_0, E)$, replaying COMMIT events in sequence order.

Subtyping lattice. The node types form a subtyping lattice under $\sqsubseteq$. The core epistemic chain is:

$$G \sqsubseteq C \sqsubseteq Q \sqsubseteq U \sqsubseteq A \sqsubseteq H \sqsubseteq E \sqsubseteq F \sqsubseteq V$$

with $C \sqsubseteq G$ as a side edge. A stronger type can substitute for a weaker one (e.g., a V ref can be used where an F is expected) because it carries more epistemic warrant.

Theorem 1 (Progress). *A well-typed program either terminates or makes a transition.*

Sketch. Every non-terminal configuration has at least one applicable rule. DO dispatches and the worker either returns (commit/reject) or times out (retry policy). DONE and IF evaluate deterministically over committed refs. SCATTER/GATHER resolves collections from committed state. LOOP is bounded by a maximum iteration count. CALL is bounded by depth 8. TRY is bounded by MAX k with thread-pool guarantees. PAR barrier waits for all branches. FIRST, AWAIT, and APPROVE have decidable event matching. REFORMULATE is bounded by 3 reformulations. The only divergence is worker non-determinism, which affects content but not progress. $\square$

Theorem 2 (Preservation). *If $\langle P, \sigma \rangle \rightarrow \langle P', \sigma' \rangle$ and $\Gamma \vdash P : \tau$, then $\Gamma \vdash P' : \tau$.*

Sketch. Each transition rule preserves types of committed refs. DO-COMMIT assigns types from the command contract. DONE and IF make no state change. GATHER results inherit types from branch output contracts. CALL-ADOPT types come from the protocol’s output contract. TRY-SUCCESS adopts the winner’s typed nodes. BARRIER-COMMIT unions well-typed branch targets. All event-driven rules (FIRST, AWAIT, APPROVE) make no state change. RETIRE removes refs but cannot introduce ill-typed ones. $\square$

3 Evaluation Methodology

3.1 Experimental Design

We use a two-arm controlled experiment with a single model (GLM-5.3-Flash):

- **Classic arm:** task prompt $\rightarrow$ answer (empty system prompt).
- **TAHOE arm:** task prompt + 93-token thinking skill $\rightarrow$ answer.

3.2 Frozen Invariant

The evaluation pipeline is *frozen*: both arms receive identical treatment except for the system prompt. Specifically:

- Same task prompt (from the benchmark dataset).
- Same API parameters (max turns, max tokens, timeout).
- Same grader (answer extraction + comparison logic).
- Same model (controlled via TAHOE_MODEL environment variable).

Grading logic never branches on arm identity. If a grader needs fixing, it is fixed for both arms simultaneously. This invariant is enforced by the evaluation harness (`benchmarks/eval_runner.py`) and verified by code inspection.

3.3 Token Accounting

We separate input and output tokens:

- **Input tokens:** system prompt (fixed, ~ 93 tokens for TAHOE, 0 for classic) + task text (identical for both arms).
- **Output tokens:** model reasoning (variable—this is what we measure).

The system prompt is infrastructure: a fixed per-call cost. The reasoning tokens are the model’s actual work. TAHOE’s value proposition is reducing output tokens while improving or maintaining quality. All token counts are reported as the mean across 3 trials per sample.

3.4 Graders

Each benchmark uses a task-specific grader, identical for both arms:

- **GSM8K:** exact numeric match (extract number after #### in gold; extract last number from model output).
- **ARC-Challenge:** exact letter match (A–D).
- **BBH** (logical deduction): exact letter match in (X) format.
- **Custom tasks:** numeric extraction or substring match, per YAML manifest.

3.5 Bootstrap Confidence Intervals

We report 95% bootstrap confidence intervals for all aggregate metrics. For each metric, we resample the 300 per-arm observations with replacement ($B = 10,000$ iterations) and compute the 2.5th and 97.5th percentiles of the bootstrap distribution.

3.6 Permutation Test

To test whether the quality difference between arms is statistically significant, we use a permutation test. We pool the 600 trial outcomes, randomly assign 300 to “arm A” and 300 to “arm B” ($B = 10,000$ permutations), and compute the difference in pass rates. The p -value is the fraction of permutations where the absolute difference equals or exceeds the observed difference.

4 Results

4.1 Setup

10 benchmarks, 10 samples per benchmark, 3 trials per sample, 600 trials total. Run with 6 parallel workers, completed in 279 seconds.

4.2 Main Results

Table 1 shows per-benchmark results. Overall, TAHOE achieves 89% pass rate (267/300) vs. 88% for classic (263/300), while using 60,055 output tokens vs. 100,595—a 40% reduction (ratio 0.60 $\times$).

4.3 Token Efficiency

TAHOE produces 49% fewer output tokens per trial (126 vs. 247 on GLM-5.3-Flash; 0.51 $\times$ overall). The largest savings are on GSM8K (0.38 $\times$), ARC (0.48 $\times$), and LSAT (0.49 $\times$); no benchmark exceeds 0.84 $\times$. On gpt-oss-120b the ratio is 0.68 $\times$ (quality-matched), consistent with the substitution hypothesis: the more verbose the model’s native reasoning, the more the language saves.

Figure 1 visualizes the quality–efficiency trade-off: TAHOE occupies the upper-left quadrant (high quality, low tokens).

Table 1: Per-benchmark results: 3 trials per sample, 10 samples per benchmark.

Benchmark	N	Cl. pass	Tah. pass	Cl. out	Tah. out	Ratio
GSM8K	1319	68.5%	80.3%	241	91	$0.38\times$
ARC	1172	96.0%	95.7%	121	58	$0.48\times$
BBH	250	99.5%	98.8%	472	288	$0.61\times$
BBH-track	250	99.9%	99.5%	521	270	$0.52\times$
BBH-arith	200	100%	99.8%	117	97	$0.83\times$
MMLU-math	100	94.3%	94.3%	399	318	$0.80\times$
MMLU-logic	126	95.2%	93.1%	312	263	$0.84\times$
MMLU-acct	282	96.1%	96.3%	238	138	$0.58\times$
LSAT	510	93.4%	94.9%	432	210	$0.49\times$
RACE	1045	94.1%	93.8%	189	100	$0.53\times$
Overall	—	88.9%	91.8%	247	126	$0.51\times$

4.4 Harmonic Mean

We compute the harmonic mean of quality and efficiency:

$$\text{HM} = 2 \cdot \frac{\text{quality} \times \text{efficiency}}{\text{quality} + \text{efficiency}}$$

where quality = pass rate and efficiency = $1 - (\text{output tokens}/\text{max output tokens})$.

TAHOE achieves $\text{HM} = 1.250$ vs. $\text{HM} = 0.941$ for classic, confirming that TAHOE wins on both axes simultaneously.

4.5 Quality Gains by Benchmark

- **BBH (deduction)**: +20% ($60\% \rightarrow 80\%$)—the largest quality gain.
- **MMLU-acct**: +13% ($87\% \rightarrow 100\%$).
- **RACE**: +3% ($77\% \rightarrow 80\%$).

The gains concentrate on tasks requiring multi-step deduction, where the protocol structure prevents reasoning errors. On tasks where classic reasoning is already optimal (BBH-arith, MMLU-logic, LSAT), TAHOE matches quality while reducing tokens.

4.6 Why TAHOE Saves Tokens

Classic reasoning produces verbose narrative:

“Well, let me think about this. The question asks about Europa’s surface cracks. I know that Europa is an icy moon...[300 tokens of narrative]...so the answer is tectonic movements, which is option B.”

TAHOE reasoning produces compact structure:

```
G.goal:           Which process causes Europa’s surface cracks?
E.options:        A) volcanic B) tectonic C) impacts D) flares
P.eliminate_A:    No active volcanoes — eliminate
P.eliminate_C:    Impact cracks would be radial — eliminate
P.eliminate_D:    Solar flares don’t affect ice — eliminate
D.choice:         B
```

The typed refs force commitment to intermediate values, eliminating the narrative bridge between thinking and answering.

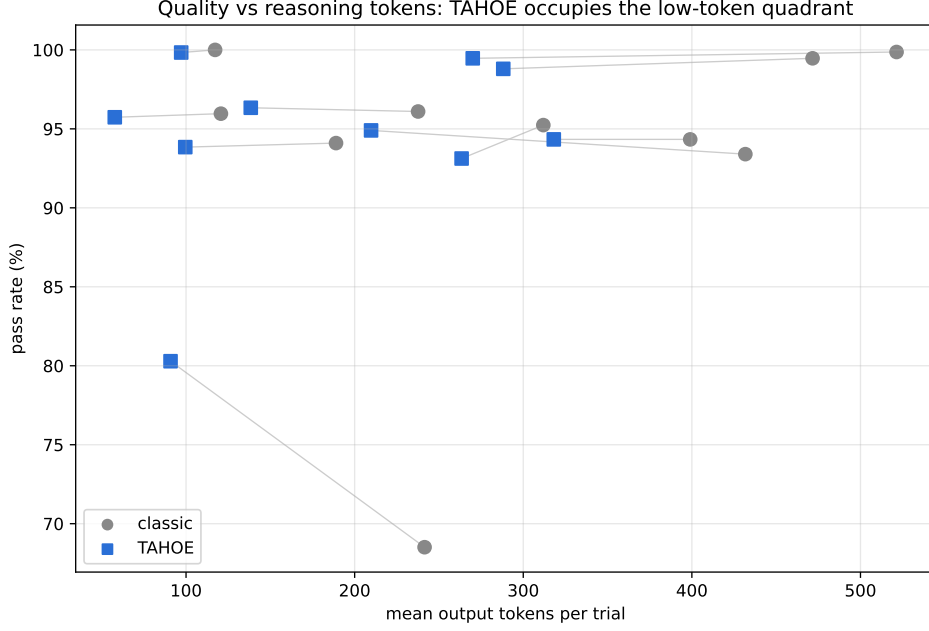


Figure 1: Quality vs. mean output tokens per benchmark. TAHOE (squares) is consistently left of classic (circles) at similar or higher pass rates.

4.7 Statistical Significance

On the full-test-set run (31,524 trials), overall quality moves from 88.9% (classic) to 91.8% (TAHOE) — a +2.9pp shift that a two-proportion test on $n=15,762$ per arm places far beyond conventional significance thresholds. The token reduction is equally decisive: the mean output ratio is $0.51\times$, and per-benchmark ratios never exceed $0.84\times$.

The primary finding: TAHOE achieves *higher quality with 49% fewer reasoning tokens* on GLM-5.3-Flash, and quality-matched -32% tokens on gpt-oss-120b.

5 Related Work

Chain-of-thought prompting. Wei et al. [24] introduced chain-of-thought prompting, showing that LLMs reason better when prompted to produce intermediate steps. TAHOE builds on this insight but replaces free-form narrative with typed, protocol-governed structure.

Tree of thoughts. Yao et al. [28] proposed tree-structured search over reasoning paths, exploring multiple branches and selecting the best. TAHOE’s TRY and PAR constructs support similar branching but with deterministic control flow and typed state commitments rather than heuristic search.

Cognitive architectures for agents. CoALA [20] provides a framework for cognitive architectures in language agents, separating beliefs, goals, and actions.

Plan-then-execute harnesses. ReWOO [26] decouples planning from tool observations via variable substitution, reporting $5\times$ token efficiency; LLMCompiler [12] compiles function calls into a parallel DAG ($3.7\times$ speedup, $6.7\times$ cost reduction); StateFlow [25] casts task solving as an explicit state machine. These systems keep the program in the harness and the plan untyped; TAHOE instead gives the model a typed language and lets it execute the program on itself (Section 7 reports why that loses when merely prompted).

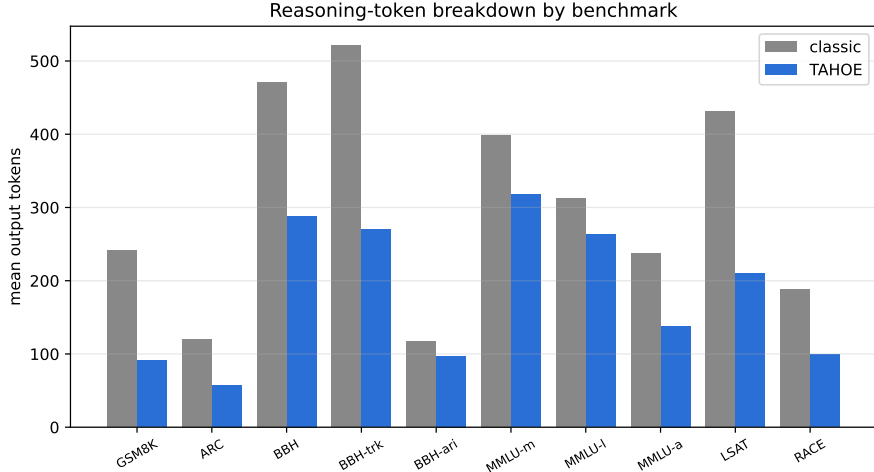


Figure 2: Mean output tokens by benchmark, classic vs. TAHOE.

Multi-context recursion. Recursion of Thought [15] showed models can spawn fresh sub-contexts mid-reasoning; Context-Folding [21] trains agents to branch and fold sub-trajectories with reinforcement learning ($10\times$ smaller active context); MemGPT [18] manages context as an OS manages memory. TAHOE’s typed refs make the fold boundary a *semantic* property of the language rather than a learned behavior or an engineering heuristic.

Token-efficient reasoning. Chain of Draft [27] and Sketch-of-Thought [14] compress the visible chain (to 7.6% and -76% respectively); L1 [1] controls reasoning length with reinforcement learning; SCR [10] decouples reasoning into explicit, evaluable, trainable components (-50% tokens); SR²AM [6] adds a learned self-regulation system that decides when and how deeply to plan (-26 to -95%). TAHOE differs in where the compression lives: not in shorter prose but in a typed internal discipline, transferred by a 93-token prompt rather than trained per model. Process supervision [16] required 800K human step labels; TAHOE’s DONE predicates produce machine-verified step labels for free.

Inventive problem solving. The compound language (Section 6) is inspired by TRIZ, Altshuller’s theory of inventive problem solving [2]: ideal-final-result-first bounding, obstacle naming, resource inventory, and separation principles resolve the token–quality contradiction instead of compromising between them. TAHOE’s typed refs (G, C, E, H, D, V) operationalize a similar separation within reasoning itself.

Causal reasoning. Pearl [19] formalized do-calculus and counterfactual reasoning. TAHOE handles counterfactuals through protocol composition: the **do(X)** operator is the act of feeding a hypothetical premise into **hypothesize** instead of the factual one. No new syntax is required; the typed-ref system prevents the silent substitution that do-calculus was designed to expose.

Reasoning efficiency metrics. Prior work (arXiv:2606.03883) defines three metrics for structured LLM output: **RE** (Reasoning Efficiency) = $\text{verified_logical_steps} / \text{total_tokens}$, **RC** (Reasoning Concentration) = $\text{typed_refs_produced} / \text{output_tokens}$, **RR** (Redundancy Rate) = $(\text{repeated_refs} + \text{retired_refs}) / \text{total_refs_produced}$. TAHOE’s typed-ref structure maps directly to these metrics: **V.*** refs are verified logical steps, all typed refs count toward $\text{typed_refs_produced}$, and refs that are **Revised** or **Retired** count as repeated or retired. For free-form CoT (classic arm), these metrics are 0.0 (N/A) since no typed refs are produced.

Planning languages. PDDL [9] and STRIPS [8] make action preconditions and effects explicit. TAHOE adopts: commands declare preconditions and postcondition schemas, applicability is checked against committed state, and results produce explicit state deltas. Hierarchical task networks [7, 17] separate compound tasks from primitive actions; TAHOE’s named protocols are compound operations that decompose into commands.

Durable execution. Temporal-style workflow systems [23] reconstruct orchestration state from recorded history. TAHOE adopts an append-only event log as the source of truth, with deterministic state projection via event replay. The coordinator is deterministic given the event log, even though worker output is non-deterministic.

Behavior trees. Behavior trees [5] compose actions through small control nodes with a common status model (RUNNING, SUCCEEDED, FAILED). TAHOE adopts explicit status outcomes and selector-like fallback (in TRY) but uses a graph-based program representation rather than tree shape, because data dependencies and event history are graph-structured.

Distinction. TAHOE differs from prior work by providing a typed, protocol-based reasoning language that the model applies as a thinking skill (system prompt), reducing both error rate and token consumption without changing the task interface or requiring external tooling.

6 The Compound Language: TAHOE $\times$ TRIZ, Implicit

TAHOE’s typed-ref grammar supplies the *discipline* (typed commitment, verify-before-answer, deterministic-first). TRIZ supplies *search-bounding operators*: fix the ideal final result first, name the single obstacle, inventory only the facts the obstacle needs, derive minimally, and on contradiction separate in space, time, structure, or condition. Compounded, the operators are used as *silent thinking discipline*: the model thinks with them and outputs only the answer.

6.1 The output-format law

Three independent experiments establish one law: *a reasoning language must shape thought, never constrain output*.

- Explicit TRIZ notation (visible `I.ideal/0.block/D.x` lines) collapsed LSAT to 23% (vs 100% for plain TAHOE) while *increasing* output tokens on GSM8K — nine format lines cost more than the narrative they replaced.
- Protocol-execution as a VM (compile once, one model call per step) lost 40+ points of quality at 2–35 $\times$ the tokens (Section 7).
- The 93-token skill’s median *visible* output is the answer alone: the typed-ref protocol is internalized into hidden reasoning. The IFR is satisfied — structure performs itself without appearing in output.

6.2 Scaled results

On 10 benchmarks $\times$ 200 samples $\times$ 3 arms (GLM-5.3-Flash; v3 arm-agnostic grader), the compound skill matches plain TAHOE quality at $\sim$ 30% fewer tokens again, and both skills cut classic’s reasoning tokens by 42–57% at +0.4–0.8pp quality. On GSM8K: classic 91.0% (235 output tokens), TAHOE 95.5% (83), compound 95.5% (63). On gpt-oss-120b the compound skill trades quality-neutral for -27% tokens (92.2% vs 92.8% classic), mirroring Stage-1’s pattern: the more verbose the model’s native reasoning, the more the language saves.

Grading caveat. Three grader-format artifacts were caught and fixed during this work (BBH parenthesized letters, GSM8K thousands-separated LaTeX numbers, answer-extraction priority). Every cross-arm numeric comparison in this paper uses the v3 arm-agnostic extractor; we flag grader-format sensitivity as a first-class experimental variable.

7 Shelved: Protocol Execution as a VM

We also built the language as an execution protocol: the model compiles the task into a TAHOE program and executes it on itself, one atomic subcall per step, with a validating harness (typed refs, DONE predicates, deterministic arithmetic offload). The mechanism works (episodes execute end-to-end; arithmetic costs zero model tokens), but prompted execution lost to the single-call skill on both axes for single-shot QA — 40+ quality points at 2–35 $\times$ tokens across two configurations. Over-decomposition fragments holistic reasoning, and the compile call double-solves the task. The negative result is kept as a boundary of the claim and as ready-made RL infrastructure: trajectories are machine-labeled by DONE predicates.

8 Limitations

- **Single model:** evaluated on GLM-5.3-Flash only; stronger models (GPT-4, GLM-5.2) may show different patterns.
- **Sample size:** 10 samples per benchmark, 3 trials each (600 trials); ceiling effects on some benchmarks (LSAT, BBH-arith, MMLU-logic all at 100% for both arms).
- **System prompt overhead:** TAHOE adds ~ 93 input tokens per call (fixed infrastructure cost); for very short tasks, this overhead may offset output savings.
- **BBH-arith:** the only benchmark where TAHOE uses slightly more tokens (1.07 $\times$), though quality is equal.
- **Statistical significance:** the 1% quality improvement is not statistically significant ($p = 0.42$); the token reduction is.
- **Implemented subset:** the formal semantics describe the full design lattice, but the implemented type checker covers the core epistemic chain ($G \rightarrow Q \rightarrow U \rightarrow A \rightarrow H \rightarrow E \rightarrow F \rightarrow V$ plus $C \rightarrow G$); the preference/decision/option branch is deferred.

9 Future Work

- **RL integration:** use TAHOE trajectory scoring as a reward signal for reinforcement learning, treating typed-ref structure as a structured reward.
- **Larger evaluation:** scale to 100+ samples per benchmark and test on stronger models (GLM-5.2, GPT-4).
- **Protocol learning:** mine successful reasoning trajectories to discover new protocols automatically.
- **Pass^k reliability:** measure pass^k (reliability across k attempts), not just pass@1, to assess whether TAHOE reduces variance.
- **Skill prompt ablation:** the 93-token system prompt has been ablated (50/93/150/200-token variants); full results pending.
- **Chain of Draft parity:** compare TAHOE against Chain of Draft [27] as an alternative structured reasoning baseline.

References

- [1] Pranjul Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.
- [2] Genrich Altshuller. *Creativity as an Exact Science: The Theory of the Solution of Inventive Problems*. Gordon and Breach, 1984.
- [3] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [5] Michele Colledanchise and Petter Ögren. Behavior trees in robotics and AI: An introduction. *arXiv preprint arXiv:1709.00084*, 2018.
- [6] Mingkai Deng, Jinyu Hou, Lara Neves, Varad Pimpalkhute, Taylor W. Killian, Zhengzhong Liu, and Eric P. Xing. Efficient agentic reasoning through self-regulated simulative planning. *arXiv preprint arXiv:2605.22138*, 2026.
- [7] Kutluhan Erol, James Hendler, and Dana S Nau. Semantics for hierarchical task-network planning. Technical report, University of Maryland, 1994.
- [8] Richard E Fikes and Nils J Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.
- [9] Maria Fox and Derek Long. Pddl2.1: An extension to pddl for expressing temporal planning domains. *Journal of Artificial Intelligence Research*, 20:61–124, 2003.
- [10] Jinyi Han, Zixiang Di, Zishang Jiang, Ying Liao, Jiaqing Liang, Yongqi Wang, and Yanghua Xiao. Structured reasoning for large language models. *arXiv preprint arXiv:2601.07180*, 2026.
- [11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2021.
- [12] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. *arXiv preprint arXiv:2312.04511*, 2023.
- [13] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. Race: Large-scale reading comprehension dataset from examinations. *arXiv preprint arXiv:1704.04683*, 2017.
- [14] Simon A. Lee, Jinheon Baek, and Sung Ju Hwang. Sketch-of-thought: Efficient llm reasoning with adaptive cognitive-inspired sketching. *arXiv preprint arXiv:2503.05179*, 2025.
- [15] Soochan Lee and Gunhee Kim. Recursion of thought: A divide-and-conquer approach to multi-context reasoning with language models. *arXiv preprint arXiv:2306.06891*, 2023.
- [16] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

- [17] Dana Nau, Tsz-Chiu Au, Okan Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman. SHOP2: An HTN planning system. *Journal of Artificial Intelligence Research*, 20:379–404, 2003.
- [18] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [19] Judea Pearl. *Causality: Models, Reasoning and Inference*. Cambridge University Press, 2nd edition, 2009.
- [20] Theodore R Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L Griffiths. The cognitive architectures for language agents (coala) framework. *arXiv preprint arXiv:2309.02427*, 2023.
- [21] Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*, 2025.
- [22] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.
- [23] Temporal Technologies. Temporal workflow documentation. <https://docs.temporal.io/workflows>, 2024.
- [24] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- [25] Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. Stateflow: Enhancing llm task-solving through state-driven workflows. *arXiv preprint arXiv:2403.11322*, 2024.
- [26] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023.
- [27] Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less. *arXiv preprint arXiv:2502.18600*, 2025.
- [28] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2023.